Tournament Management Platform

Master Plan, Part 4 of 4: Dependencies, Milestones, Accounts, Postponed Tasks, Pitfalls and Master Checklist

1. Phase dependency overview

Phase	Depends on	Can run in parallel with	Blocks
A Requirements	-	-	Everything
B Repo and tools	A	C (account creation)	C, D, E
C Atlas	A, B16	B	D (db connect)
D Backend foundation	B, C	E	G and all server features
E Frontend foundation	B, D6 (API contract)	D	All UI work
F DB design	A	D, E	G onward (models)
G Auth and admin	D, E, F	-	All role-protected features
H Public shell	G	-	I
I Tournaments	G, H	-	J, K, N
J Registrations	I	U (notifications model)	K
K Captains and teams	J	-	L, N
L Auction config	K	-	M
M Live auction	L	N, O, P	V (realtime polish)
N Knockout fixtures	K	M	P, R
O League fixtures	N1	M	P
P Groups and standings	N, O	M	R, T
Q Manual fixtures	N	M, P	R
R Manual results	P, Q	-	S, T, X
S CSV import	R and a real CSV sample	T	Final stats
T Corrections	R, P	S	X
U Notifications	J (models), then all event sources	M-T	W
V Realtime hardening	M, U	W	AD
W Dashboards	J, K, U, Z	V	AB
X Archives	R, T	Y	AD
Y Exports	P, R, X	X	AD
Z Privacy and lifecycle	G, J, M	W	AD
AA Audit viewing	All audited features	AB	AD
AB Polish	W	AA	AD
AC Testing	Starts at D, continuous	All	AD
AD Deployment	Everything above (staging deploy can begin after M)	-	Launch

2. Milestones and acceptance criteria

Milestone	Includes	Acceptance criteria
M1 Foundations	A to G	Repo on GitHub with CI green; backend and frontend talk; DB connected; users register/login; refresh keeps session; admin exists and cannot be self-created; organizer apply/approve works; permission matrix tests pass; audit rows written.

Milestone	Includes	Acceptance criteria
M2 Public + tournaments	H, I	Visitors browse, search, filter without login; approved organizer creates and configures tournaments; rule edits recorded; lifecycle transitions validated; invite-only tournaments listed publicly.
M3 Registration	J, K	No duplicate or over-capacity registrations under parallel load; approvals, waiting list, withdrawals work; captains, eligibility and manual teams enforce squad limits.
M4 Auction	L, M, V (core)	Parallel bids cannot corrupt state; countdown is server-controlled and survives restart; undo appends events; purse/squad never violated; visitors watch live but cannot bid; finalization validates all squads.
M5 Fixtures + standings	N, O, P, Q	All three formats generate correctly (tests for many sizes); manual fixtures/scheduling audited; standings only from confirmed results; qualification and bracket progression correct.
M6 Results	R, S, T	Manual and CSV results previewed and confirmed; unconfirmed data never public; corrections preserve originals and recalc downstream; malformed CSV rejected safely.
M7 Product complete	U, W, X, Y, Z, AA, AB	Notifications persist and update live; all dashboards complete; archives public; exports safe (formula injection test); privacy and deletion workflows verified; audit viewers scoped correctly; responsive and accessible.
M8 Production	AC, AD	Public URL works; WebSocket auction verified in production; backups restored successfully; monitoring alerts tested; release checklist all ticked; v1.0.0 tagged.

3. External accounts, credentials, files and decisions to prepare

Accounts

Item	When needed	Notes
GitHub account (+ 2FA)	Phase B	Private repo recommended; enable branch protection
MongoDB Atlas account (+ 2FA)	Phase C	Separate project/cluster for production later
Frontend host account	Phase AD (staging earlier)	Cloudflare Pages / Netlify / Vercel (verify limits)
Backend host account	Phase AD (staging after M)	Render / Railway / Fly.io / VPS (verify WebSocket and sleep behaviour)
Error monitoring (e.g. Sentry)	Phase AD	Free tier usually enough
Uptime monitor	Phase AD	Pings /api/health; also reduces cold starts on some hosts
Domain registrar (optional)	Phase AD	Helps with same-site cookies; not mandatory
Password manager	Phase A	Store all credentials; never in chat or Git

Credentials/secrets you will generate (never commit)

- MONGODB_URI (dev, prod)
- SESSION_SECRET / cookie signing secret
- ADMIN_EMAIL and one-time ADMIN_BOOTSTRAP_PASSWORD
- CLIENT_ORIGIN allow-list
- Monitoring DSN keys
- Backup storage credentials

Files and decisions

- Real sample scorecard CSV(s), sanitized (Phase S blocker)
- Sport/game name and typical scoring concepts (for standings defaults)
- Answers to Part 1 issue table (rounds, invites, etc.)
- Logo/branding/colors (optional, before Phase AB)
- Privacy/retention policy wording
- Support contact method for password recovery
- Hosting budget ceiling

4. Tasks that must wait for a prerequisite

Task	Wait until	Reason
CSV parser/final stats model	Real CSV sample and format spec (S1, S2)	Do not invent columns
Production deployment	M7 complete; hosting limits re-checked	Free tiers change; avoid rework
Impact-preview real calculations	Phase P	Needs standings engine
Notification wiring	Phase U service exists	Use hooks/stubs until then
Auction UI	Auction engine tests pass (M12)	Engine first, UI second
Seed demo data for auction	Teams and eligibility (K)	Needs real structure
Charts/statistics pages	Stats fields known (S)	Show only supported stats
PDF design polish	Data models stable (after T)	Avoid rewriting layouts
Custom domain	Topology decision (AD2)	Cookie behaviour depends on it
Load testing	Phase V	Needs realtime in place
Redis/multi-instance scaling	Only if usage proves it necessary	Single instance suffices; avoid overengineering

5. Common setup mistakes and when to avoid them

Mistake	Stage	Avoidance
Committing .env or Atlas URI	B5, C6	Create .gitignore before first commit; check git status; rotate any leaked secret immediately
Atlas open to 0.0.0.0/0 permanently	C4, AD4	Allow specific IPs; document any temporary exception and remove it
Special characters in Atlas password break URI	C6	Generate URL-safe password or URL-encode
Using the Atlas owner account in the app	C3	Dedicated least-privilege DB user
Installing dependencies in the wrong folder	B15	Always cd client or cd server first; separate lockfiles
Mixing Tailwind v3 and v4 setup instructions	E1	Check installed version; follow its docs only
CORS wildcard with credentials	D6, AD9	Explicit origin allow-list; credentials true only for those
Cookies not sent in production (cross-site)	AD2	Same-site topology or proxy; test on Safari/mobile early
Trusting client-side checks	Every phase	Server validates role, owner, state, amounts
Testing on a standalone Mongo (no transactions)	D13	Use replica-set test DB
Transactions that do network calls or run long	J, M	Keep them short and DB-only
Float money/prices	F, L	Integers only
Auction state in memory only	M	Persist endsAt and state; recover at boot
Client clocks for countdown	M9	Render from server time offset
Seeding script runnable in production	C11, AD7	Guard by NODE_ENV and require explicit flag
SPA refresh gives 404 in production	AD8	Rewrite all routes to index.html
Free host sleeps during auction	AD1, AD10	Pick persistent host; warm and monitor; schedule accordingly
Line-ending and path problems on Windows	B3	autocrlf=input, .gitattributes, short paths, repo outside OneDrive
CSV export formula injection	Y2	Neutralize leading = + - @ tab and carriage return
Leaking unconfirmed scores in public API	R2	Serializer returns scores only for confirmed results; test it
Skipping backup restore test	AD13	Restore into scratch DB before launch

6. Master implementation checklist

Tick each step as you complete it. Step IDs match Part 2. Do not skip the "Done when" verification in the roadmap.

Phase A: Requirement finalization and preparation

- [ ] **A1** Re-read the Blueprint and this plan; list every requirement with an ID (R-001...) in docs/requirements.md
- [ ] **A2** Answer the open issues from Part 1 section 1.2
- [ ] **A3** Confirm stack: TypeScript, monorepo, server-side sessions, extra collections
- [ ] **A4** Define MVP slicing: M1 to M8 milestones (Part 4); mark nothing as dropped
- [ ] **A5** Define module boundaries and which module owns which collection
- [ ] **A6** Draft state-machine tables for tournament, registration, lot, fixture, result, account request
- [ ] **A7** Decide hosting shortlist and domain approach (same-site cookies)
- [ ] **A8** List accounts/services needed (Part 4 section 3) and sign up for GitHub and Atlas

Phase B: Development environment and GitHub repository

- [ ] **B1** Check/install Node.js LTS, npm, Git for Windows, VS Code; record versions
- [ ] **B2** Install VS Code extensions: ESLint, Prettier, Tailwind CSS IntelliSense, MongoDB for VS Code (optional), GitLens (optional)
- [ ] **B3** Configure Git identity, default branch main, autocrlf=input, and a SSH key or GitHub credential manager
- [ ] **B4** Create project folder C:\dev\tournament-platform and open in VS Code
- [ ] **B5** git init; create root .gitignore (node_modules, .env*, dist, coverage, logs, .DS_Store) and .gitattributes/.editorconfig
- [ ] **B6** Create root README.md (purpose, prerequisites, run commands placeholder)
- [ ] **B7** Make the first commit on main (docs + ignores only)
- [ ] **B8** Create the GitHub repository (private recommended), no auto-generated files
- [ ] **B9** Connect remote (origin) and push main
- [ ] **B10** Set branch strategy: main (always deployable) + short-lived feature/* branches + PRs; Conventional Commit messages; protect main
- [ ] **B11** Decision recorded: one repo for client and server (as per Blueprint)
- [ ] **B12** Create client/ and server/ and docs/ folders (with .gitkeep)
- [ ] **B13** Initialize server app: npm init in server/, set type=module, add TypeScript, tsx, @types/node, tsconfig; do not install runtime deps yet
- [ ] **B14** Initialize client app: npm create vite (react-ts) inside client/
- [ ] **B15** Install dependencies separately in server/ and client/ (never mix lockfiles)
- [ ] **B16** Create .env.example in server/ and client/ with placeholder values only; copy to .env locally
- [ ] **B17** Add ESLint + Prettier configs; scripts lint, format, typecheck per app
- [ ] **B18** Add root package.json with convenience scripts (dev via concurrently, lint, test, build)
- [ ] **B19** Run both apps and confirm they work (Vite page loads; server placeholder logs listening)
- [ ] **B20** Commit and push skeleton in logical commits; add GitHub Actions CI stub (lint + typecheck)

Phase C: MongoDB Atlas and database initialization

- [ ] **C1** Create MongoDB Atlas account and organization
- [ ] **C2** Create Atlas project "tournament-platform" and a free/lowest-cost cluster in a nearby region
- [ ] **C3** Create a dedicated DB user for dev with a long generated password; least privilege (readWrite on dev DB only)
- [ ] **C4** Configure Network Access: add your current IP only (avoid 0.0.0.0/0 except documented temporary need)
- [ ] **C5** Choose database names: tournament_dev, tournament_test (memory), tournament_prod (later, separate cluster/user)
- [ ] **C6** Copy the connection string (SRV) into server/.env as MONGODB_URI (URL-encode special chars in password)
- [ ] **C7** Install mongoose and zod; write env validation (fail fast on missing vars)
- [ ] **C8** Connect Mongoose and log success; handle connection errors/retry
- [ ] **C9** Test: connection check script + /health endpoint reporting db state (after D)
- [ ] **C10** Plan transactions: confirm replica-set support (Atlas shared tier is replica set); list transactional operations (Part 3)
- [ ] **C11** Plan seed strategy: scripts/seed.ts (idempotent, dev only, refuses to run in production) + demo data
- [ ] **C12** Plan admin provisioning: scripts/create-admin.ts reads ADMIN_EMAIL and one-time password from environment, creates the only admin, aborts if one exists
- [ ] **C13** Plan dev/prod separation (separate clusters, users, secrets, DB names; NODE_ENV guards)
- [ ] **C14** Plan backups and restores: mongodump schedule, off-provider storage, restore drill; versioned migrations (migrate-mongo or simple scripts with schemaVersion field)

Phase D: Backend initialization and foundation

- [ ] **D1** Install runtime deps: express, mongoose, zod, helmet, cors, cookie-parser, express-rate-limit, pino, pino-http, socket.io, argon2 or bcryptjs
- [ ] **D2** Install dev deps: vitest, supertest, mongodb-memory-server, tsx, types

- [ ] **D3** Create folder structure (config, middleware, modules, sockets, scripts, utils)
- [ ] **D4** Create app.ts (Express factory) and server.ts (listen + graceful shutdown)
- [ ] **D5** Add config/env.ts with Zod validation and typed env access
- [ ] **D6** Add middleware: helmet, CORS allow-list from env, JSON body limit, cookie parser, request id, pino-http logging
- [ ] **D7** Define API response and error conventions: {data} / {error:{code,message,details}}; AppError classes; central error handler; 404 handler
- [ ] **D8** Add Zod validation middleware (body, query, params)
- [ ] **D9** Add rate-limit middleware presets (auth, bids, admin, general)
- [ ] **D10** Add /api/health and /api/ready endpoints
- [ ] **D11** Set up session model + auth/role middleware shells (authenticate, requireRole, requireOwner) returning 401/403
- [ ] **D12** Configure Socket.IO on the same HTTP server; cookie-based socket auth; namespaces /auction and /live (no handlers yet)
- [ ] **D13** Set up Vitest, Supertest and in-memory replica set helper; write first tests (health, error handler)
- [ ] **D14** Verify first endpoint through browser/Postman/curl
- [ ] **D15** Commit; add backend README section (scripts, env vars)

Phase E: Frontend initialization and foundation

- [ ] **E1** Configure Tailwind correctly for the installed major version; verify with a utility class
- [ ] **E2** Install react-router, TanStack Query, react-hook-form + zod resolver, socket.io-client
- [ ] **E3** Create folder structure (app, components, pages, features, services, hooks, types)
- [ ] **E4** Configure Vite env files (VITE_API_URL, VITE_SOCKET_URL) and a dev proxy to the API so cookies are same-origin in development
- [ ] **E5** Build centralized API client (fetch wrapper: credentials, JSON, error normalization, 401 handling)
- [ ] **E6** Create router with route groups: public, auth, player, organizer, admin; route guards driven by /auth/me
- [ ] **E7** Create auth context/store (session bootstrap on refresh, logout)
- [ ] **E8** Build layouts (public shell with header/footer, dashboard shell with sidebar) with responsive nav
- [ ] **E9** Build design-system primitives: Button, Input, Select, Modal, Table, Badge, Toast, Tabs, Pagination, Skeleton
- [ ] **E10** Build state components: Loading, Empty, Error, Unauthorized/Forbidden, NotFound
- [ ] **E11** Build form helpers with Zod schemas shared/duplicated with server types
- [ ] **E12** Wire frontend to backend: page calls /api/health and displays status
- [ ] **E13** Add Vitest + React Testing Library setup; first component tests

Phase F: Database design finalization

- [ ] **F1** Take the 15 Blueprint collections and evaluate each against requirements
- [ ] **F2** Evaluate additional collections (sessions, ruleRevisions, invites, auctionSessions, importJobs, resultRevisions, counters)
- [ ] **F3** Decide embed vs reference for every relationship (growth, query pattern, history, concurrency)
- [ ] **F4** Define unique/compound/partial indexes for integrity rules
- [ ] **F5** List transaction-sensitive workflows and their atomic units
- [ ] **F6** Draw ERD (Mermaid in docs/erd.md)
- [ ] **F7** Fix schema implementation order (Part 3 section 9)
- [ ] **F8** Review design against workflow walkthroughs (registration, bid, undo, correction, delete account)

Phase G: Authentication, authorization and administrative bootstrap

- [ ] **G1** Create User model (normalized unique email, passwordHash, role, status, visibility settings) and Session model with TTL index
- [ ] **G2** Implement password hashing service (Argon2id or bcrypt) with policy (min length, breached-pattern guard)
- [ ] **G3** Implement register (player) and login with generic error messages, throttling per IP and per email
- [ ] **G4** Implement session issue/validate/rotate; httpOnly Secure SameSite cookie; CSRF defence (Origin check + custom header)
- [ ] **G5** Implement logout and logout-all; revocation when status becomes suspended
- [ ] **G6** Implement /auth/me and frontend bootstrap so refresh keeps login
- [ ] **G7** Implement role-based authorization middleware and tournament-ownership guard
- [ ] **G8** Create admin bootstrap script (single admin, env-supplied one-time password, forces password change, refuses duplicates); block admin creation via public API
- [ ] **G9** Implement OrganizerApplication model, submit endpoint, admin approve/reject with reason, status change notification hook
- [ ] **G10** Implement account suspension (admin) with session revocation
- [ ] **G11** Implement AccountRequest model and admin-assisted recovery: user request -> admin verifies -> single-use short-lived token (hash stored) -> user sets password -> sessions revoked
- [ ] **G12** Implement account deletion request (request only; processing in Phase Z)
- [ ] **G13** Implement AuditLog model + append-only service (no update/delete API) and log all G actions
- [ ] **G14** Frontend: register, login, logout, organizer application, admin approval queue (basic)
- [ ] **G15** Permission test suite: visitor, player, unapproved organizer, approved organizer, admin versus every protected route

Phase H: Public website shell and tournament discovery

- [ ] **H1** Tournament model (minimal read fields) + seed demo tournaments
- [ ] **H2** Public API: list with pagination, search, filters (status, format, date), sort
- [ ] **H3** Public API: tournament detail with section endpoints (overview, rules, teams, fixtures, standings, announcements)
- [ ] **H4** Homepage: upcoming, ongoing, recently completed, featured/updated; search bar
- [ ] **H5** Discovery page with filter UI and URL-synced query params
- [ ] **H6** Tournament detail page with tabs and deep links (/t/:id/fixtures)
- [ ] **H7** Invite-only tournaments shown publicly with a registration-restricted badge
- [ ] **H8** Component + API tests; responsive check

Phase I: Tournament creation, configuration, rules and lifecycle

- [ ] **I1** Complete Tournament model: basics, format, registration settings, rules object, status, schemaVersion
- [ ] **I2** Zod schemas for create/update with format-specific rule presets and sensible defaults
- [ ] **I3** Lifecycle state machine service (Draft -> RegistrationOpen -> RegistrationClosed -> TeamFormation -> AuctionInProgress -> FixturesPublished -> Ongoing -> Completed; Cancelled from allowed states)
- [ ] **I4** Create/edit endpoints with ownership check; organizer-only draft editing UI
- [ ] **I5** TournamentRuleRevision model; every rule edit records old/new/actor/time/reason
- [ ] **I6** Impact-preview service (dry-run) for rule changes affecting fixtures, standings, qualification; hooks stubbed until those phases
- [ ] **I7** Auction-parameter freeze guard (issue 1)
- [ ] **I8** Registration deadline automation: timer + lazy check on access; reopen endpoint with audit
- [ ] **I9** Cancel and reschedule endpoints (reason required) + audit + notification hook
- [ ] **I10** Organizer UI: wizard, rules editor with history view, lifecycle controls

Phase J: Player registration, approvals, waiting lists and withdrawal

- [ ] **J1** Registration model with partial unique index on (tournament, player) for active statuses; counters doc for capacity
- [ ] **J2** Invite model + invite-only gate (code/link redemption)
- [ ] **J3** Register service in a transaction: eligibility, deadline, duplicate, capacity -> approved / pending / waitlisted with atomic waiting position
- [ ] **J4** Auto vs manual approval; organizer approve/reject with reason
- [ ] **J5** Waiting list: organizer promotes player manually when a spot opens; audit each change
- [ ] **J6** Withdrawal rules (pre-auction / pre-start), organizer exceptions, historical retention
- [ ] **J7** Reopen registration interplay with waiting list
- [ ] **J8** Notification hook calls (stub until U)
- [ ] **J9** Player UI (register, status, withdraw) and organizer UI (queue, waiting list)
- [ ] **J10** Integration + concurrency tests

Phase K: Captain applications, selection, auction eligibility, team formation, manual assignment

- [ ] **K1** Add captain-interest fields to registration; player toggle during registration
- [ ] **K2** Team model + team creation (count, names); counters for squadCount and purse
- [ ] **K3** Organizer selects captains (final decision) and links captain to team; audit
- [ ] **K4** Auction-eligibility decision endpoint (separate from captain selection)
- [ ] **K5** TeamAssignment model with history (method, actor, previous)
- [ ] **K6** Manual assignment service in a transaction enforcing squad limit, eligibility, uniqueness
- [ ] **K7** Publish team assignment notifications; public teams/squads tab
- [ ] **K8** Organizer UI for captains, eligibility, drag/assign teams
- [ ] **K9** Tests: duplicate assignment, squad limit, ineligible player

Phase L: Auction configuration and session initialization

- [ ] **L1** AuctionSession model: purses, squad max, base price, increment, countdown, order, unsold policy, status, round counter, version
- [ ] **L2** Configuration endpoints and validation (all values numeric, positive, consistent with squad and pool sizes)
- [ ] **L3** Auction order: fixed list / random seeded (store seed) / manual
- [ ] **L4** Start endpoint: validates, freezes config, creates AuctionLot per eligible player, initializes team purses, sets tournament status AuctionInProgress
- [ ] **L5** Organizer config UI with dry-run summary
- [ ] **L6** Tests for config validation and initialization

Phase M: Live auction interface and engine

- [ ] **M1** AuctionLot and AuctionBid models (append-only bids, compensating undo events, roundNumber, outcome)

- [ ] **M2** Bid service: single atomic conditional update (status, version, currentPrice) inside transaction; validates captain, round rule, increment X+Y, purse, squad capacity
- [ ] **M3** Done endpoint: organizer closes round, server sets countdown endsAt
- [ ] **M4** Countdown engine: server timer, persisted endsAt, auto-sell on expiry, cancel on bid/undo, recovery at boot, lazy check on access
- [ ] **M5** Undo-last-bid: compensating event restoring price/leader, cancels countdown, requires reason
- [ ] **M6** Sell / unsold handling, purse and squad updates, TeamAssignment creation in same transaction
- [ ] **M7** Unsold round endpoint and finalize endpoint with full squad validation
- [ ] **M8** Socket.IO broadcasting of lot state to /auction room; snapshot on connect; reconnect resync
- [ ] **M9** Public auction room UI (read-only), captain bid UI, organizer control panel; display server-offset countdown
- [ ] **M10** Rate limits on bids; rejection messages for stale/conflicting bids
- [ ] **M11** Admin-only post-finalization correction path (audited)
- [ ] **M12** Full auction test suite (concurrency, countdown, undo, purse, squad) + simulated auction script

Phase N: Knockout fixture generation

- [ ] **N1** Fixture model with stage, round, slots, dependency refs (winnerOf/loserOf), status
- [ ] **N2** Bracket generator: seeding modes (random with stored seed, manual), byes for non-power-of-two
- [ ] **N3** Dependency graph resolution: next-round slots populated when confirmed results arrive
- [ ] **N4** Manual-override record for bracket edits
- [ ] **N5** Bracket UI (public + organizer)

Phase O: League and round-robin fixture generation

- [ ] **O1** Circle-method generator (single/double round robin, odd teams with bye)
- [ ] **O2** Home/away balance and round numbering
- [ ] **O3** League table view hooking into standings (P)

Phase P: Groups, standings, qualification and knockout progression

- [ ] **P1** Group draw (random seed stored / manual) and group fixtures
- [ ] **P2** Standings engine: pure function from confirmed results + rules; points presets; configurable tie-breakers (head-to-head, differential, etc. as supported)
- [ ] **P3** Qualification evaluator (top N per group, best-of-rest options) with ties surfaced to organizer
- [ ] **P4** Qualification override with previous/new/reason; audit and snapshot
- [ ] **P5** Populate knockout fixtures after qualification confirmed
- [ ] **P6** standingsSnapshots written on every confirm/correct/override/rule change
- [ ] **P7** Wire I6 impact preview to real standings/fixture calculations
- [ ] **P8** UI: standings tables, group views, qualification badges

Phase Q: Manual fixtures, scheduling, rescheduling and cancellation

- [ ] **Q1** Manual fixture create/edit with validation (teams belong to tournament, no impossible pairs)
- [ ] **Q2** Schedule editing (date, time, venue) with reason on changes; timezone handling
- [ ] **Q3** Postpone/cancel fixture statuses and their effect on progression
- [ ] **Q4** Participant notifications for fixture changes (hooks)
- [ ] **Q5** Organizer schedule UI (calendar/list)

Phase R: Match status and manual result entry

- [ ] **R1** MatchResult + ResultRevision models (draft/previewed/confirmed, immutable revisions)
- [ ] **R2** Match status transitions (scheduled/ongoing/completed/postponed/cancelled); public sees status only
- [ ] **R3** Manual result entry with validation and preview
- [ ] **R4** Confirm endpoint: transaction updates result, fixture, standings, bracket, snapshot, audit
- [ ] **R5** Organizer UI for status and result entry
- [ ] **R6** Tests: confirmation atomicity, public data leakage

Phase S: CSV scorecard upload, parsing, validation, preview and confirmation

- [ ] **S1** Obtain a real representative scorecard CSV (and ideally 2 to 3 variants)
- [ ] **S2** Document the column map, types, and meanings from the sample
- [ ] **S3** Build importer adapter interface (parse -> normalize -> validate -> map) with the first adapter for the spec
- [ ] **S4** Upload endpoint: type/size validation, in-memory parse, ImportJob with preview and row errors
- [ ] **S5** Preview UI highlighting errors, unsupported fields and inline fixes
- [ ] **S6** Confirm step: promote ImportJob data to MatchResult; supported stats only; unreviewed imports never affect standings

[ ] **S7** Fuzz and malformed tests (encoding/BOM, delimiters, huge, formula chars)

Phase T: Result correction, history, standings and qualification recalculation

[ ] **T1** Correction endpoint requiring reason; new immutable revision; original preserved
[ ] **T2** Recalculate standings and qualification in the same transaction; compare snapshots
[ ] **T3** Downstream fixture/bracket impact detection with organizer confirmation if bracket would change
[ ] **T4** Correction UI with impact preview and history timeline
[ ] **T5** Tests for chains of corrections and bracket impacts

Phase U: Announcements and persistent notifications

[ ] **U1** Notification model with indexes (recipient, read, created) and TTL/retention policy
[ ] **U2** Central notify() service with event catalogue (Blueprint section 11 list) and recipient resolution
[ ] **U3** Replace hooks from I, J, K, M, Q, R, T with notify() calls
[ ] **U4** Announcement model, publish/pin/history, notify participants
[ ] **U5** Notification API (paginated, mark read, mark all read) + UI bell and page

Phase V: Socket.IO real-time updates and fallback

[ ] **V1** Namespaces/rooms: auction:, tournament:, user:; auth on handshake; authorization per room
[ ] **V2** Events: lot state, bids, countdown ticks (end time based), fixture/status/result published, notifications
[ ] **V3** Reconnection: client resync via REST snapshot; version numbers to detect gaps
[ ] **V4** Polling fallback via TanStack Query with configurable interval (env/admin setting)
[ ] **V5** Load test with many read-only spectators

Phase W: Dashboards

[ ] **W1** Player dashboard: registrations, statuses, teams, fixtures, notifications, privacy settings
[ ] **W2** Organizer dashboard: owned tournaments, queues, auction, fixtures/results, announcements, private activity log, exports
[ ] **W3** Admin dashboard: organizer queue, users, all tournaments, requests, suspended items, audit logs, interventions

Phase X: Tournament archives and historical match pages

[ ] **X1** Finalize endpoint: validates all results, freezes tournament, writes final snapshot, publishes archive
[ ] **X2** Public archive list and tournament history pages
[ ] **X3** Historical match detail page with revision history (public parts only)

Phase Y: Tournament reports and CSV/PDF exports

[ ] **Y1** Export service with permission checks and history-visibility filtering
[ ] **Y2** CSV exports: squads, fixtures, standings, results, supported stats, auction summary; neutralize formula injection (prefix = + - @ tab)
[ ] **Y3** PDF exports on demand (streamed), pagination and fonts
[ ] **Y4** Export UI and size limits/rate limits

Phase Z: Privacy controls, account lifecycle, suspensions and administrative workflows

[ ] **Z1** History-visibility setting honored by all public endpoints and exports
[ ] **Z2** Deletion processing: anonymize user shell, scrub email/credentials, keep IDs on results/audit; admin resolves request
[ ] **Z3** Organizer suspension/removal of a player with reason and side effects on eligibility, teams, fixtures
[ ] **Z4** Admin user management, tournament suspension, exceptional interventions (each audited with reason)
[ ] **Z5** Recovery workflow hardening and UI

Phase AA: Audit-log viewing and administrative oversight

[ ] **AA1** Admin audit viewer (filter by actor, action, entity, date; pagination)
[ ] **AA2** Organizer private tournament log (scoped to own tournaments)
[ ] **AA3** Audit coverage review against Blueprint section 16

Phase AB: Accessibility, responsive design, performance and final polish

[ ] **AB1** Responsive pass for all pages at 360, 768, 1280 widths
[ ] **AB2** Accessibility pass (labels, focus, contrast, ARIA for live auction region, reduced motion)
[ ] **AB3** Performance pass (indexes, pagination, bundle splitting, image sizes, Lighthouse)
[ ] **AB4** Final UI polish, copy review, empty/error states

Phase AC: Testing and quality assurance (introduced throughout)

- [ ] **AC1** Unit tests (Vitest): pure logic such as hashing, state machines, standings, bracket and round-robin generators
- [ ] **AC2** Backend API tests (Supertest) incl. validation, error format, rate limits
- [ ] **AC3** Database integration tests on in-memory replica set (transactions, unique indexes)
- [ ] **AC4** Role and ownership permission matrix (visitor, player, unapproved organizer, approved organizer, other organizer, admin)
- [ ] **AC5** Authentication and session tests (cookies, expiry, revocation, CSRF, brute force)
- [ ] **AC6** Registration capacity and duplicate tests under parallel requests
- [ ] **AC7** Auction concurrency, simultaneous bids, countdown, undo, purse, squad-limit tests
- [ ] **AC8** Fixture generation tests for knockout, round-robin, groups (sizes, byes, odd counts)
- [ ] **AC9** Result confirmation and correction tests (atomicity, downstream effects)
- [ ] **AC10** Standings and qualification recalculation tests (ties, corrections)
- [ ] **AC11** CSV validation and malformed-file tests
- [ ] **AC12** Socket.IO connection, auth, room and reconnection tests
- [ ] **AC13** Frontend component and interaction tests (React Testing Library)
- [ ] **AC14** End-to-end tests (Playwright): register/login, organizer apply, create tournament, register, auction smoke, result confirm
- [ ] **AC15** Responsive and accessibility checks (axe, keyboard, 360px)
- [ ] **AC16** Security testing: OWASP checklist, dependency audit, authz bypass attempts, upload abuse, header review
- [ ] **AC17** Backup and restoration test
- [ ] **AC18** Load/soak sanity for auctions and spectators

Phase AD: Production deployment

- [ ] **AD1** Re-check current free/low-cost offerings of shortlisted hosts (frontend, backend, Atlas) and record limits: cold starts, bandwidth, storage, connections, execution time, backup policy
- [ ] **AD2** Decide URL topology so cookies work: same-site domain/subdomains, or reverse proxy, or serve built client from the API
- [ ] **AD3** Create production accounts/projects (host, Atlas prod project, optional domain, monitoring)
- [ ] **AD4** Provision production Atlas cluster, dedicated prod DB user, restrict network access to host egress IPs or required ranges
- [ ] **AD5** Create production secrets (session secret, DB URI, admin bootstrap password, CORS origins) in provider secret manager
- [ ] **AD6** Deploy backend: build with tsc, start script, PORT binding, trust proxy, health check path
- [ ] **AD7** Run one-time production init: index sync + create-admin script (never seed demo data)
- [ ] **AD8** Deploy frontend: build with production API URL, SPA fallback rewrite to index.html for deep links
- [ ] **AD9** Configure CORS/allowed origins, cookie flags (Secure, SameSite), Socket.IO transports and sticky/single-instance assumptions
- [ ] **AD10** Verify Socket.IO in production (WebSocket upgrade, idle timeouts, reconnect) with a mock auction
- [ ] **AD11** Production smoke test of auth, uploads, reports, and every major workflow (use checklist in Part 4)
- [ ] **AD12** Set up logs, error monitoring (e.g. Sentry free tier), uptime/health monitor, alerts
- [ ] **AD13** Create backup job (mongodump to private storage), retention, and perform a restore into a scratch database
- [ ] **AD14** GitHub Actions deploy workflow: on merge to main run tests -> deploy; manual approval for production; rollback instructions
- [ ] **AD15** Final release checklist (Blueprint section 24) and security review

Milestone sign-offs

- [ ] M1 reached (acceptance criteria met)
- [ ] M2 reached (acceptance criteria met)
- [ ] M3 reached (acceptance criteria met)
- [ ] M4 reached (acceptance criteria met)
- [ ] M5 reached (acceptance criteria met)
- [ ] M6 reached (acceptance criteria met)
- [ ] M7 reached (acceptance criteria met)
- [ ] M8 reached (acceptance criteria met)